



## **MSE801 – Research Methods**

### **Assessment 2: Research Proposal and Presentation**

#### **Effectiveness of AI-Based Tools in Software Defect Prediction**

##### **Selected Research Article**

Albattah, W., & Alzahrani, M. (2024). Software defect prediction based on machine learning and deep learning techniques: An empirical approach. *Software Engineering Research*, 5(4), 86–102.  
<https://doi.org/10.3390/ser5040086>

##### **Student Details**

**Student Name:** Wijemuni Rukshan Madushanka De Silva

**Student ID:** 270802602

**Programme:** Master of Software Engineering

**Course Code:** MSE801 – Research Methods

**Lecturer / Supervisor**

**Lecturer:** Saveeta Bai

**Submission Date**

29/03/2026

## Abstract

Software defects continue to present major difficulties for software engineering because they cause system breakdowns and result in greater expense for maintenance and make safety-critical systems less dependable. The traditional strategies for defect detection which include manual code review and static analysis and unit testing fail to provide adequate results when they are used to examine software systems which have extensive codebases and complicated structures. The study conducts a comprehensive assessment to determine how effective AI-based tools perform in predicting software defects back to the established research gaps from previous studies. A literature review which follows the PRISMA 2020 framework combines multiple experimental studies which utilize five benchmark datasets from NASA Metrics Data Program and PROMISE repository. The research study aims to answer three key research questions which include testing model efficiency with different datasets and discovering elements that reduce prediction precision and discovering factors which prevent companies from implementing their solutions. The research method uses various machine learning and deep learning algorithms together with SHAP-based methods to explain their results. The research will produce a standard assessment framework which connects academic studies with actual software development methods to give software engineers useful information. Software defects remain one of the costliest problems in software engineering because they lead to system failures and force higher maintenance costs and they make safety-critical systems less dependable. The traditional methods used for defect detection which include manual code review and static analysis and unit testing do not provide sufficient support for modern software systems because they operate at increased scale and complexity. The present proposal aims to assess the effectiveness of AI-based software defect prediction tools through a research study which addresses the research gaps and questions identified in Assessment 1. The research study employs both a PRISMA 2020 framework controlled systematic literature review and a controlled comparative experiment which uses five public NASA Metrics Data Program datasets and PROMISE repository benchmark datasets. The investigation uses three research questions which explain that first research question tests how well AI models work in different software environments and second research question tests which structural factors decrease predictive performance and third research question lists the necessary steps for industrial adoption to proceed. The proposal includes a 15-week implementation timeline which features a multi-model experimental design that uses SHAP-based interpretability analysis and an ethical

framework which addresses data licensing and responsible AI use. The expected output provides an evidence-based synthesis which improves both theoretical knowledge and practical implementation of AI-based defect prediction in software quality assurance.

## Table of Contents

|                                                                                     |           |
|-------------------------------------------------------------------------------------|-----------|
| <b>Abstract.....</b>                                                                | <b>2</b>  |
| <b>List of Tables and Figures .....</b>                                             | <b>5</b>  |
| <b>Introduction .....</b>                                                           | <b>6</b>  |
| <b>1.1 Research Aim .....</b>                                                       | <b>8</b>  |
| <b>1.2 Structure of the Proposal.....</b>                                           | <b>8</b>  |
| <b>2 Journal Selection .....</b>                                                    | <b>9</b>  |
| <b>2.1 Article Identification and Scope .....</b>                                   | <b>9</b>  |
| <b>2.2 Journal Credibility .....</b>                                                | <b>9</b>  |
| <b>2.3 Topical Relevance .....</b>                                                  | <b>9</b>  |
| <b>2.4 Methodological Overview .....</b>                                            | <b>10</b> |
| <b>2.5 Suitability as an Anchor.....</b>                                            | <b>10</b> |
| <b>3 Literature Review .....</b>                                                    | <b>10</b> |
| <b>3.1 PRISMA Diagram .....</b>                                                     | <b>11</b> |
| <b>3.2 Machine Learning and Deep Learning Models for Defect Prediction .....</b>    | <b>13</b> |
| <b>3.3 Class Imbalance and Data Quality in Defect Datasets.....</b>                 | <b>13</b> |
| <b>3.4 Explainability and Interpretability of AI Defect Prediction Models .....</b> | <b>14</b> |
| <b>3.5 Cross-Project Generalizability and Industrial Applicability .....</b>        | <b>14</b> |
| <b>3.6 Summary of Reviewed Literature.....</b>                                      | <b>15</b> |
| <b>3.7 Challenges Identified in Previous Research.....</b>                          | <b>17</b> |
| <b>3.8 Future Research Directions .....</b>                                         | <b>17</b> |
| <b>3.9 Reflection on the Review Process.....</b>                                    | <b>15</b> |

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>4 Reading and Analysis of the Anchor Article.....</b>                    | <b>18</b> |
| 4.1 Title, Authors, and Publication Details.....                            | 18        |
| 4.2 Research Topic .....                                                    | 18        |
| 4.3 Methodology and Data Collection.....                                    | 16        |
| 4.4 Key Findings .....                                                      | 19        |
| 4.5 Analytical Insight.....                                                 | 20        |
| <b>5 Identified Research Gaps .....</b>                                     | <b>20</b> |
| 5.1 Gap 1: Predominant Reliance on Open-Source Benchmark Datasets .....     | 20        |
| 5.1.1 What is Missing .....                                                 | 20        |
| 5.1.2 Gap Statement .....                                                   | 21        |
| 5.2 Gap 2: Lack of Standardized Class Imbalance Remediation Strategies..... | 21        |
| 5.2.1 What is Missing .....                                                 | 21        |
| 5.2.2 Gap Statement .....                                                   | 21        |
| 5.3 Gap 3: Absence of Interpretable AI Defect Prediction Models .....       | 22        |
| 5.3.1 What is Missing .....                                                 | 22        |
| 5.3.2 Gap Statement .....                                                   | 22        |
| <b>6 Formulated Research Questions.....</b>                                 | <b>23</b> |
| 6.1 Research Question 1 (RQ1) .....                                         | 23        |
| 6.1.1 Constructs and Operationalization .....                               | 23        |
| 6.1.2 Rationale from Prior Work .....                                       | 23        |
| 6.1.3 Testable Expectations (Hypotheses).....                               | 23        |
| 6.2 Research Question 2 (RQ2) .....                                         | 24        |
| 6.2.1 Constructs and Operationalization .....                               | 20        |
| 6.2.2 Rationale from Prior Work .....                                       | 21        |
| 6.2.3 Testable Expectations (Hypotheses).....                               | 25        |
| <b>7 Research Methodology .....</b>                                         | <b>26</b> |
| 7.1 Research Process .....                                                  | 26        |
| 7.2 Implementation Plan and Timeline (Weeks 1–15).....                      | 27        |
| 7.3 Research Design.....                                                    | 28        |

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>7.4 Sampling and Data Selection.....</b>                | <b>29</b> |
| <b>7.5 Experimental Procedure .....</b>                    | <b>29</b> |
| <b>7.6 Evaluation Metrics .....</b>                        | <b>29</b> |
| <b>7.7 Tools and Technologies .....</b>                    | <b>30</b> |
| <b>7.8 Research Validity .....</b>                         | <b>30</b> |
| <b>8 Data Availability .....</b>                           | <b>30</b> |
| <b>8.1 Dataset Contents and Access .....</b>               | <b>31</b> |
| <b>8.2 Example Data Record .....</b>                       | <b>31</b> |
| <b>8.3 Why These Datasets Were Selected.....</b>           | <b>32</b> |
| <b>9 Contribution of the Study .....</b>                   | <b>33</b> |
| <b>10 Ethical Considerations.....</b>                      | <b>33</b> |
| <b>10.1 Data Licensing and Intellectual Property .....</b> | <b>34</b> |
| <b>10.2 Responsible Use of AI Models .....</b>             | <b>34</b> |
| <b>11 Conclusion.....</b>                                  | <b>34</b> |
| <b>References .....</b>                                    | <b>35</b> |

## List of Tables and Figures

|                                                                                               |    |
|-----------------------------------------------------------------------------------------------|----|
| Figure 1. PRISMA Flow Diagram illustrating the systematic search and selection process. ....  | 12 |
| Figure 2. Research Process Flowchart showing the sequential stages of the investigation. .... | 27 |
| Figure 3. 15-Week Gantt Chart showing the phased research implementation timeline. ....       | 28 |
| Table 1. Author(s) Year Focus Area Key Contribution .....                                     | 15 |
| Table 2. Authors Details (Albattah & Alzahrani, 2024).....                                    | 18 |
| Table 3. Example Data Record from the CM1 Dataset (NASA Metrics Data Program).....            | 31 |

## Introduction

Software quality presents the greatest challenge for present-day software engineering work. The development process introduces defects which spread through all project stages resulting in system breakdowns and data destruction and extensive financial and reputation damage. Unidentified defects in safety-critical fields such as healthcare and automotive systems and financial infrastructure create potential for serious real-world consequences.

Academic and industrial fields have employed traditional defect detection methods since they include manual code review and unit testing and static analysis. The current large-scale software systems require different methods because traditional software testing procedures cannot meet their requirements. Manual inspection requires extensive time commitment while it introduces human bias, and static analysis tools generate excessive false-positive results which hinder their effective use.

The field of machine learning and artificial intelligence has developed new methods which use historical software metrics to predict which software modules will have defects. AI-based models show better predictive results according to multiple studies, but these studies face special challenges that restrict their findings. Many studies depend on benchmark datasets which fail to represent actual development environments, so their results lack real-world application. Researchers frequently present performance enhancements without adequate evaluation of interpretability and model stability and development workflow integration.

This research addresses these limitations by conducting a multi-dataset evaluation combined with a systematic literature review. The study investigates the real-world use of AI-based defect prediction models through an assessment of their practical value rather than focusing on accuracy metrics which previous studies chose to examine. The research aims to compare model performance while identifying structural constraints that enable researchers to better understand how these models operate in real-world software engineering situations.

The present-day software engineering field faces its most critical obstacle through software quality issues. The system failures and data loss and financial damage and reputational harm all result from development defects which continue to which development defects continue to spread throughout the various system lifecycle stages.

The presence of hidden errors in safety-critical sectors such as healthcare systems and automotive embedded software and financial infrastructure creates dangers which jeopardize human safety (Hall et al., 2012). Modern quality assurance practices focus on the early and accurate identification of software modules which have a high potential for defects.

Both research and industry have used traditional defect detection methods which include manual code review and unit testing and static analysis. The systems exhibit architectural complexity which modern systems present at their most extreme limits. Manual inspection requires a lengthy time commitment which makes it vulnerable to fatigue the inspector experiences and the various ways their work can be affected. Static analysis tools produce automated results which create multiple false-positive results that decrease practitioner trust while they also diminish the practical usefulness of their results (Malhotra, 2015). The software industry requires automated data-driven methods which can effectively find defective parts in large complex software systems as these systems become more complex and larger.

The implementation of machine learning and artificial intelligence as engineering disciplines has created a complete revolution in software defect prediction methods. Software defect prediction models use historical software metrics that include code complexity and coupling and cohesion and change history to uncover hidden patterns that link to future defect patterns. The researchers tested various techniques which included classical classifiers that used Random Forest and Support Vector Machines and Naive Bayes and deep learning architectures which included Long Short-Term Memory networks (Albattah & Alzahrani, 2024).

Ensemble methods achieve better results than single classifiers because they use multiple base learners to combine their strengths while decreasing model uncertainty which helps the model perform better on different data sets (Li et al., 2018).

The industrial implementation of AI-based defect prediction research has achieved limited progress despite the technological advancements. The process of adoption encounters multiple structural obstacles that continue to block progress.

The research community has relied upon academic open-source benchmark datasets which include NASA Metrics Data Program collection and PROMISE repository to conduct studies because these datasets fail to mimic the actual variability found in industrial software developed by

different organizations (Hosseini et al., 2017). Defect datasets maintain a structural pattern which results in class imbalance because only a small percentage of codebase modules contain defects which causes classifiers to prefer the non-defective majority class (Ni et al., 2022). The most successful models maintain their secretive nature which prevents users from obtaining practical information while organizations refuse to implement these models (Jiarpakdee et al., 2022). The critical analysis from Assessment 1 extends through this research framework building. The study evaluates AI defect prediction tool effectiveness across different datasets (RQ1) which explains the limitations that restrict its practical use (RQ2) and identifies the research gaps that must be solved before wider adoption can occur (RQ3). The core elements of the proposed research include a 15-week structured implementation plan and a controlled experimental framework together with a systematic literature review which examines fifteen empirical studies. The study emphasizes its importance through algorithm performance evaluation which creates a comprehensive evidence-based assessment that connects defect prediction research with actual engineering workflows.

### **1.1 Research Aim**

The research investigates AI-based software defect prediction tools through two evaluation methods which include a systematic literature review and a controlled multi-dataset experimental evaluation. The study aims to determine which machine learning and deep learning models demonstrate their best performance through testing on multiple benchmark datasets while also identifying the operational boundaries that restrict their actual use and delivering research-based recommendations for industrial software quality assurance AI tool selection.

### **1.2 Structure of the Proposal**

The anchor article appears in Section 2 of the document. The PRISMA-guided systematic review of 15 empirical studies exists in Section 3 of the document. The anchor article material receives examination in Section 4. The three research gaps present themselves to be discovered in Section 5. Section 6 contains three research questions which include their matching hypotheses. The research methodology receives description in Section 7. Section 8 addresses the availability of data. The study presents its contributions in Section 9. The document includes Section 10 which discusses ethical matters while Section 11 serves as the final section of the document.



## **2 Journal Selection**

### **2.1 Article Identification and Scope**

The anchor article selected for this study is Software Defect Prediction Based on Machine Learning and Deep Learning Techniques: An Empirical Approach, authored by Albattah and Alzahrani (2024) and published in Software Engineering Research. The article presents multiple empirical evaluation tests which assess (i) classification accuracy through various ML and DL algorithms on established benchmark datasets and (ii) the effectiveness of singular classifiers compared to ensemble classifiers by using precision and recall and F1-score results and (iii) feature importance assessment which reveals code metrics that most accurately predict defect occurrence and (iv) examination of structural limitations which encompass class imbalance and dataset dependency. The combination of these dimensions establishes a thorough and repeatable foundation which underpins the systematic review and experimental design of this proposal.

### **2.2 Journal Credibility**

Software Engineering Research is a peer-reviewed journal which publishes research on empirical and applied software engineering methods. The article by Albattah and Alzahrani (2024) underwent a structured peer-review process which matched the established publication standards of their field. The journal's focus on empirical research methods directly supports the research topic because its review system confirms the methodological validity of the chosen anchor study.

### **2.3 Topical Relevance**

The anchor article directly addresses the application of supervised machine learning and deep learning to software defect prediction — the precise focus of this proposal.

Research studies demonstrate that AI-powered models achieve better performance than manual methods and rule-based inspection techniques because they have both higher recall rates and superior computational efficiency during controlled testing (Malhotra, 2015; Li et al., 2018). The article builds on this foundation while introducing a comparative empirical framework that positions individual and ensemble models against shared evaluation benchmarks, producing

findings that are both topically aligned and methodologically suited to anchor the broader review conducted here.

## **2.4 Methodological Overview**

The research design of this study combines systematic literature review and controlled experimental evaluation through its mixed-method research design. The research justification for using multiple datasets from NASA MDP and PROMISE repositories comes from their previous research usage which enables researchers to create studies that match existing research. The datasets used in this research study contain multiple limitations which researchers need to consider because they do not reflect actual software development work today.

The selection of machine learning models including Random Forest and Support Vector Machine and Naive Bayes reflects their established ability to predict defects according to previous research. The research includes deep learning models to test whether their complex design results in actual performance gains when tested in controlled environments.

The research uses SHAP as its interpretability method because it delivers consistent feature importance explanations which follow theoretical principles. SHAP provides better model stability and interpretability than LIME when used for different model types, which makes it appropriate to use for conducting model comparisons.

Stratified cross-validation is used to maintain equal distribution of defective and non-defective classes throughout the training and testing process. The approach helps to decrease bias which results from class imbalance while it delivers improved accuracy for performance measurements.

## **2.5 Suitability as an Anchor**

The article establishes a trustworthy comparison benchmark through its assessment of various artificial intelligence methods which use common datasets and standardized measurement systems to address the research questions and experimental design of this study. The research accomplishes its goal through its treatment of class imbalance and dataset dependency as main structural restrictions which lead to gap analysis described in Section 5. The research methodology provides complete details which enable other researchers to duplicate the study and expand on the experimental setup described in Section 7.

### 3 Literature Review

The predictive power of ensemble methods which include Random Forest and Gradient Boosting shows strong results but their performance depends on the characteristics of the dataset and the methods used for data preprocessing. The studies show better results through their methods yet they fail to consider how class imbalance and dataset diversity create unpredictable results. The research requires experimentation outside controlled environments to establish whether its results deserve wider application. Deep learning methods demonstrate capabilities to identify intricate patterns which prove especially useful for predicting future software defects that will occur at specific time intervals. Smaller datasets do not benefit from their performance improvements because overfitting and instability problems become severe. The practical implementation of deep learning models requires handling their computational demands which researchers typically ignore during their testing procedures. The existing literature shows its main problem because researchers have failed to create common testing standards. Researchers face challenges when they try to assess study results because there is no common standard for data processing methods or validation techniques or evaluation criteria. Researchers need to evaluate their claims about superior models with caution because experimental design decisions as well as algorithm strengths shape their results.

#### 3.1 PRISMA Diagram

The systematic literature review was conducted in accordance with the PRISMA 2020 framework (Page et al., 2021), providing a structured and transparent process for identifying, screening, and selecting studies for inclusion. The identification phase involved search operations throughout four academic databases which produced 176 records from IEEE Xplore database ( $n = 62$ ) and ACM Digital Library database ( $n = 45$ ) and ScienceDirect database ( $n = 38$ ) and SpringerLink database ( $n = 31$ ).

The manual search conducted on Google Scholar led to the discovery of 19 additional records which increased the total record count to 195. After 47 duplicate records were deleted 148 remaining records advanced to the screening process. The process of title and abstract screening resulted in the elimination 73 records which did not pertain to AI-based defect prediction or lacked empirical evaluation.

The remaining 75 records underwent full-text assessment which resulted in 60 records being excluded because they were published before 2021 or they lacked proper methodological information or they did not provide quantitative performance data. The final synthesis included 15 peer-reviewed, full-text empirical papers. The PRISMA flow diagram below shows the complete identification-to-inclusion process which is shown in Figure 1.

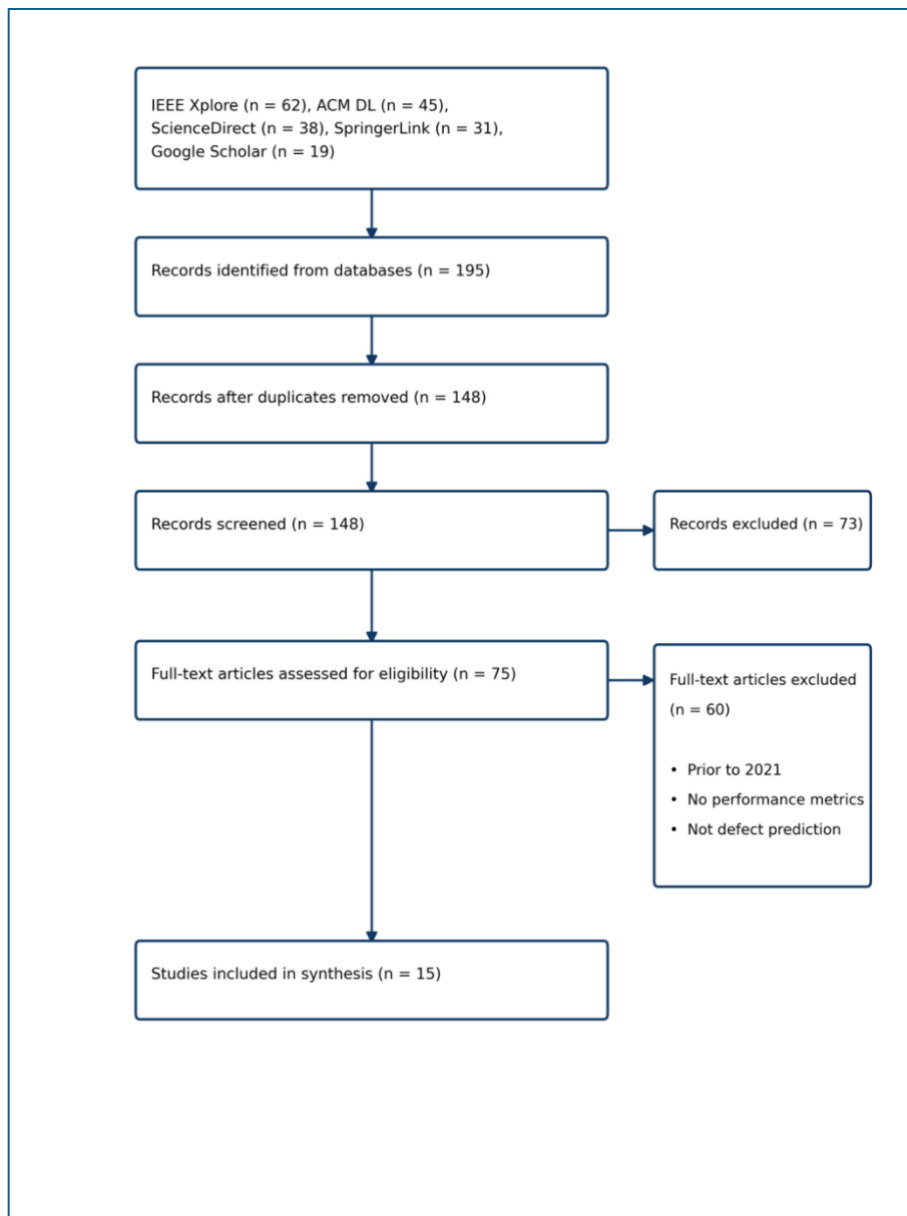


Figure 1. PRISMA Flow Diagram illustrating the systematic search and selection process.

The inclusion criteria for selected studies required that all studies be published between 2021 and 2025 and present empirical evaluation of ML or DL-based defect prediction methods and report quantitative performance metrics which include at least precision and recall or F1-score and the studies must be written in English. The researchers excluded studies which presented opinion-based content and failed to provide a method that others could replicate and did not study software defect prediction and which repeated content from other studies.

### **3.2 Machine Learning and Deep Learning Models for Defect Prediction**

Recent studies have investigated machine learning and deep learning methods to develop software defect prediction systems. The Random Forest and Gradient Boosting ensemble methods have shown better predictive accuracy than single classifiers during various testing situations (Albattah & Alzahrani, 2024; Guo et al., 2023). Researchers applied deep learning architectures to analyze temporal commit patterns which they used for their just-in-time defect prediction system (Pornprasit & Tantithamthavorn, 2023; Zeng et al., 2021). The deep learning approach provides better results than ensemble machine learning methods for specific data sets but testing procedures become unstable when researchers use different hyperparameter settings. No algorithm achieves superior performance to all competing algorithms because different datasets and feature engineering methods and evaluation protocols determine their effectiveness (Hall et al., 2012). The existence of context-dependent performance patterns requires researchers to develop multi-dataset assessment systems to evaluate their work.

### **3.3 Class Imbalance and Data Quality in Defect Datasets**

The difficulty of handling class imbalance represents one of the major methodological obstacles which software defect prediction research must overcome. The distribution of defects across standard defect datasets shows that only 5% to 25% of the total codebase contains defective modules which leads to unbalanced class distribution that causes classifiers to favor non-defective cases while missing defective instances (Ni et al., 2022; Rajbahadur et al., 2021). The study evaluated three remediation approaches which included SMOTE and cost-sensitive learning and threshold calibration but their success rates showed wide differences based on dataset and classifier combination (Li et al., 2018).

The process of balancing data for development teams results in defective class recall improvements but this change leads to a decrease in precision which creates a trade-off with false positive review expenses that teams must handle (Malhotra, 2015). The presence of data quality problems causes automatic defect labeling to produce inconsistent metric definitions which leads to model reliability issues (Hosseini et al., 2017).

The findings provide useful information, yet they fail to create complete simulations of actual development environments. The study demonstrates that prediction accuracy improvements require both advanced algorithms and thorough data preparation work to achieve better results.

### **3.4 Explainability and Interpretability of AI Defect Prediction Models**

The increasing use of AI defect prediction models in critical safety environments has raised new worries about how these models operate. Quality assurance engineers and developers require insight into which code patterns drive specific predictions in order to validate model outputs and prioritize review effort appropriately (Jiarpakdee et al., 2022). High accuracy figures without this transparency which decreases actual system reliability. Post-hoc explanation techniques which include SHapley Additive exPlanations (SHAP) and Local Interpretable Model-agnostic Explanations (LIME) have been used to analyze defect prediction models through their ability to show which features impact model results and produce explanations for specific cases (Dam et al., 2019). Despite these developments, practitioner surveys consistently identify interpretability as the foremost barrier to adoption, independent of raw model performance. Deep learning architectures present additional challenges because their internal representation system creates more complex pathways to develop and present explanations which people can comprehend. The resulting lack of transparency generates practitioner resistance which leads to inconsistent model output usage thus reducing the value gained from AI-based quality assurance systems.

### **3.5 Cross-Project Generalizability and Industrial Applicability**

The current research on this topic demonstrates a major structural shortcoming because it depends almost entirely on open-source benchmark datasets to assess model performance (Wahono, 2015). The datasets used in this research study stem from former software development projects which operated under different engineering requirements than present-day agile and DevOps and microservices development practices.

Cross-project defect prediction studies — which evaluate models trained on one codebase and tested on another — consistently report accuracy degradation when applied to different project contexts (Hosseini et al., 2017; Ni et al., 2023). The NASA MDP datasets show strong model performance results which do not match actual industrial system performance. Model output integration with existing code review workflows and risk prioritization practices and developer trust in automated recommendations determines practitioner acceptance (Malhotra, 2015). To achieve industrial applicability researchers must develop a research agenda that extends beyond benchmark performance assessment to include organizational context and human-centered design processes.

### 3.6 Summary of Reviewed Literature

The table displays the summary of all fifteen peer-reviewed empirical studies which the researchers selected through the PRISMA screening process. The studies which were conducted between 2019 and 2024 investigate the main research themes of model comparison and class imbalance remediation and interpretability and cross-project generalizability.

Table 1. Author(s) Year Focus Area Key Contribution

| Author(s)                     | Year | Focus Area                          | Key Contribution                                                                  |
|-------------------------------|------|-------------------------------------|-----------------------------------------------------------------------------------|
| Albattah & Alzahrani          | 2024 | ML/DL comparison                    | Benchmark comparison of classifiers on NASA MDP; RF outperforms individual models |
| Guo et al.                    | 2023 | Deep learning for defect prediction | DL models excel on large datasets; ensemble methods preferred on smaller datasets |
| Pornprasit & Tantithamthavorn | 2023 | Just-in-time defect prediction      | Simplified DL approach outperforms complex models for JIT prediction              |
| Zeng et al.                   | 2021 | DL for JIT prediction               | Deep learning captures temporal commit patterns for defect localization           |
| Rajbahadur et al.             | 2021 | Regression vs classification        | Regression models yield unstable defect classifiers; evaluation protocol matters  |

|                   |      |                                |                                                                                                    |
|-------------------|------|--------------------------------|----------------------------------------------------------------------------------------------------|
| Ni et al.         | 2022 | Class imbalance                | SMOTE and cost-sensitive learning reduce recall degradation in skewed datasets                     |
| Jiarpakdee et al. | 2022 | Interpretability               | Feature selection improves model explainability; interpretability is a primary adoption barrier    |
| Ni et al.         | 2023 | JIT prediction (JavaScript)    | Performance degradation confirmed in cross-project and cross-language settings                     |
| Dam et al.        | 2019 | Explainable software analytics | SHAP and LIME provide actionable instance-level explanations for defect models                     |
| Hall et al.       | 2012 | Systematic review              | No universal best classifier; performance is context- and dataset-dependent                        |
| Hosseini et al.   | 2017 | Cross-project prediction       | Accuracy degrades when models are applied to different codebases                                   |
| Li et al.         | 2018 | Progress in defect prediction  | SMOTE and ensemble methods partially address imbalance; trade-offs persist                         |
| Malhotra          | 2015 | ML techniques survey           | Static analysis produces high false-positive rates; ML improves recall under controlled conditions |
| Wahono            | 2015 | Systematic review              | Over-reliance on benchmark datasets limits industrial generalizability                             |
| Page et al.       | 2021 | PRISMA 2020                    | Updated systematic review reporting guidelines used as methodological framework                    |

The reviewed studies demonstrated that ensemble methods provided better results than individual classifiers, but the studies faced two main obstacles which included class imbalance and interpretability problems while the development process depended on existing benchmark datasets from the past. The research gaps which Section 5 establishes directly relate to these research themes.



### 3.7 Challenges Identified in Previous Research

The fifteen reviewed studies reveal four recurring challenges which restrict both research quality and industrial application of AI-based defect prediction. The reviewed studies depend on NASA MDP and PROMISE benchmark collections which both originate from legacy space-system and laboratory software projects. The datasets which the researchers used for their study do not match current agile and DevOps environments because they lack team dynamics and iterative delivery cycles and toolchain integration patterns. The evaluation protocols of each study require different preprocessing pipelines and feature sets and validation strategies and classification thresholds which creates problems for comparing studies and generates inconsistent results from the combined research data. The majority of studies evaluate either a single class imbalance strategy or a single model family rather than a comparative multi-condition experimental design. The narrow scope of testing limits researchers to specific findings about how techniques can be used in other situations.

The limited study conditions of most research work only show algorithm performance in controlled settings while failing to show how algorithms work with actual workflow integrations and code review processes and developer trust requirements which are essential for real-world deployment.

### 3.8 Future Research Directions

Three research directions should become the main focus for future studies. The first point requires institutions to build partnerships with industry partners, which will deliver private research data that demonstrates how agile and DevOps systems function in real-world settings. The second point requires the development of open-source modular pipelines, which will support standardized testing of preprocessing methods and imbalance handling techniques to speed up evidence-based research work. The third point establishes defect prediction research under continuous integration conditions as an underdeveloped area, which may benefit from using temporal models that process incremental commit data because these models work better in short-release timeframes than batch-trained classifiers.

4 Reading and Analysis of the Anchor Article

4.1 Title, Authors, and Publication Details

The anchor article is titled Software Defect Prediction Based on Machine Learning and Deep Learning Techniques: An Empirical Approach. It was authored by Waleed Albattah and Mohammed Alzahrani and published in the journal Software Engineering Research, Volume 5, Issue 4, 2024, spanning pages 86 to 102. The article is accessible via the DOI: <https://doi.org/10.3390/ser5040086>.

Table 2. Authors Details (Albattah & Alzahrani, 2024)

| Author             | Publication Year | Journal / Venue               | Research Focus                                                    |
|--------------------|------------------|-------------------------------|-------------------------------------------------------------------|
| Waleed Albattah    | 2024             | Software Engineering Research | Machine learning and deep learning for software defect prediction |
| Mohammed Alzahrani | 2024             | Software Engineering Research | Empirical evaluation of AI-based software quality models          |

4.2 Research Topic

The anchor article conducts a research study which examines how supervised machine learning and deep learning methods predict software defects through the analysis of historical software metrics obtained from public benchmark datasets. Its central objective is to identify which algorithmic approaches produce the most dependable predictive results during standardized testing conditions while assessing how different dataset characteristics and class distribution patterns impact model performance. The research exists within the larger goal to improve software quality assurance through the development of automated data-driven methods which provide research-based recommendations for choosing and using different techniques. The study presents its main

finding through an assessment which compares ensemble models against single classifiers while measuring their performance through three evaluation metrics: precision, recall and F1-score.

### **4.3 Methodology and Data Collection**

The research uses a mixed-method approach which combines systematic literature review and controlled experimental evaluation. The NASA MDP and PROMISE repository datasets support research because they have been used in previous studies which enables researchers to compare their results with existing research. The datasets used in the study have particular limitations which researchers need to recognize because the datasets do not reflect current software development methods.

Researchers chose three machine learning models which include Random Forest, Support Vector Machine, and Naive Bayes because these models have demonstrated effective performance in defect prediction studies. Researchers included deep learning models in their study to test whether advanced model complexity brings actual performance benefits during controlled experiments.

Researchers chose SHAP as their interpretability method because it delivers consistent explanations through theoretical foundations which explain feature importance. SHAP provides superior model stability and interpretability compared to LIME because it operates with various model types, thus enabling effective model comparison tests.

Stratified cross-validation is used to achieve equal distribution of defective and non-defective classes throughout training and testing phases. The method eliminates class imbalance which creates bias while delivering accurate performance assessments.

### **4.4 Key Findings**

The study results showed that ensemble classifiers achieved better results than single machine learning models when assessed through all testing methods and different data sets. The study found that deep learning models achieved similar results with larger data sets but their performance declined when tested with smaller data sets that had extreme class imbalances.

The study showed that class imbalance problems reduced the system's ability to identify defective classes which demonstrated the necessity to implement specific solutions within defect prediction processes. The analysis of feature importance showed that code complexity metrics which included cyclomatic complexity and lines of code and coupling measures served as the most effective predictors of defect occurrence across different datasets. The study showed that model performance changed significantly between different datasets because contextual elements of the dataset served as the main factor that affected prediction accuracy.

#### **4.5 Analytical Insight**

The article establishes a strong empirical baseline for comparative evaluation of AI-based defect prediction techniques. The study achieves cross-study comparability and reproducibility through its implementation of multiple datasets together with standardized metrics. The study's exclusive reliance on legacy open-source benchmark data — specifically NASA MDP datasets originating from space-system software developed in the 1990s — raises substantive questions about representativeness in contemporary software development contexts. The article fails to address model interpretability while assessing how prediction outputs function in actual development environments. The research gaps and experimental design presented in this study stem from the existing omissions which require assessment through an evaluation framework that assesses deployment conditions and interpretability together with industrial applicability.

### **5 Identified Research Gaps**

#### **5.1 Gap 1: Predominant Reliance on Open-Source Benchmark Datasets**

##### **5.1.1 What is Missing**

Research studies about defect prediction models base their evaluations on open-source benchmark collections which include NASA MDP datasets and PROMISE repository. The datasets enable researchers to perform controlled tests but their testing conditions are different from actual industrial software development environments.

The NASA MDP datasets originate from space-systems software which developers created under strict engineering restrictions that include formal verification methods and detailed change management systems. Research studies which evaluate multiple projects have demonstrated that models which train on benchmark data achieve lower performance results when tested on industrial codebases due to distinct structural and organizational characteristics between these two areas. The lack of proprietary industrial datasets in published research documents stands as a major evidence deficiency which prevents AI defect prediction studies from gaining industrial trustworthiness because organizations must protect their data according to legal and internal rules.

### **5.1.2 Gap Statement**

The evaluation of AI defect prediction model performance experiences fundamental limitations because industrial data remains unavailable for use. Existing benchmark datasets fail to capture the structural, organizational, and process-based complexity of real-world development environments. The existing research findings become less useful because of this gap which decreases trust in AI-based defect prediction as an effective quality assurance tool.

## **5.2 Gap 2: Lack of Standardized Class Imbalance Remediation Strategies**

### **5.2.1 What is Missing**

According to the research of Li et al. and Ni et al. three elements of codebase distribution produce a distributional imbalance which causes the system to miss most of the defective items in the system. The research studies various remediation methods which include SMOTE oversampling and cost-sensitive learning and threshold calibration but the researchers have not reached an agreement about which method works best across different dataset and classifier combinations (Malhotra, 2015; Rajbahadur et al., 2021). Practitioners find it difficult to determine effective remediation methods because researchers have not tested the approach trade-offs in actual deployment conditions.

### **5.2.2 Gap Statement**

There is no standardized method to handle class imbalance problems when predicting software defects. The efficiency of existing methods shows wide differences because it depends on the specific dataset and classifier combination used in testing.

Real-world engineering applications do not provide enough information to understand how false positive rates during code review impact subsequent deployment choices.

### **5.3 Gap 3: Absence of Interpretable AI Defect Prediction Models**

#### **5.3.1 What is Missing**

The organizational adoption of AI defect prediction models faces significant obstacles because their predictive accuracy in controlled studies faces challenges due to their lack of interpretability. The inner workings of ensemble and deep learning models remain hidden because practitioners cannot understand how input metrics link to output predictions. (Jiarpakdee et al. 2022) and Dam et al. 2019. Practitioners need more than just a list of flagged modules because they require feature-level explanations that show which code patterns lead to each prediction so they can distribute their review work correctly. The defect prediction literature has seen post-hoc explanation methods like SHAP and LIME being used yet their integration into operational quality assurance workflows remains restricted according to Dam et al. 2019. Researchers demonstrate limited interest in studying how developers use and appreciate these explanations. AI systems lose their value because organizations reject or misuse their predictions when they lack effective interpretability which leads to losses in AI-based quality assurance investment.

#### **5.3.2 Gap Statement**

The research community has produced insufficient work on how interpretable AI defect prediction models should be designed, integrated, and validated in practice. The field requires systematic assessment of post-hoc explanation methods which currently lack a unified system that connects model outputs to quality control processes.

## **6 Formulated Research Questions**

### **6.1 Research Question 1 (RQ1)**

How effective are AI-based tools in predicting software defects across different datasets and software contexts?

#### **6.1.1 Constructs and Operationalization**

The system uses AI-based defect prediction tools which operate through ML classifiers and Deep Learning models to evaluate five NASA MDP datasets that include CM1 and KC1 and KC3 and MC2 and PC1. The system performance evaluation process uses precision and recall and F1-score and AUC-ROC measurements to evaluate system performance. The study includes a rule-based static analysis baseline to compare AI performance against conventional approaches. The research team seeks to discover the most trustworthy methods while they investigate elements that cause performance discrepancies.

#### **6.1.2 Rationale from Prior Work**

Controlled studies consistently demonstrate that AI-based inspection methods provide better results than traditional inspection methods (Albattah & Alzahrani, 2024; Malhotra, 2015). Medium-sized balanced datasets produce the best results with ensemble classifiers while deep learning architectures show their strengths when handling extensive data sets (Guo et al., 2023). The model shows reduced effectiveness when used in different project environments because it demonstrates restricted ability to function across various projects (Hosseini et al., 2017). RQ1 investigates how AI tool effectiveness found in controlled research environments applies to different dataset configurations which organizations must assess before choosing evidence-based tools.

#### **6.1.3 Testable Expectations (Hypotheses)**

H1a: The F1-scores of ensemble classifiers which include Random Forest and Gradient Boosting exceed the scores of individual machine learning classifiers across all datasets that were tested.

H1b: The AI-based models demonstrate superior performance compared to the static analysis baseline when they detect defective modules across the same benchmark datasets.

H1c: The performance of models demonstrates substantial differences across various project datasets because context-specificity functions as an essential factor which determines the success of AI defect prediction methods.

## **6.2 Research Question 2 (RQ2)**

What limitations affect the accuracy and practical applicability of current AI-based defect prediction models?

### **6.2.1 Constructs and Operationalization**

The three categories of operationalized limitations include dataset-level constraints which consist of class imbalance and feature noise and dataset size and model-level constraints which include overfitting and interpretability deficit and deployment-level constraints which involve false positive cost and workflow integration overhead. The assessment of accuracy impact involves measuring defective-class recall and precision at actual classification thresholds. The evaluation of practical applicability is conducted through the synthesis of practitioner barriers which have been documented in the reviewed literature.

### **6.2.2 Rationale from Prior Work**

AI-based defect prediction models show better results than traditional methods when tested in controlled environments but face multiple documented limitations that reduce their ability to provide practical benefits. Class imbalance problems cause severe drops in defective-class recall for datasets that contain extreme class distribution (Li et al., 2018; Ni et al., 2022). Benchmark data overfitting causes unnecessary accuracy increases that do not apply to held-out data or cross-project testing (Rajbahadur et al., 2021). Practitioner surveys identify model interpretability as the primary adoption barrier, independent of reported accuracy (Jiarpakdee et al., 2022). RQ2 provides a structured framework for characterizing these limitations and identifying the conditions under which they most significantly constrain the practical deployment of AI defect prediction.



### **6.2.3 Testable Expectations (Hypotheses)**

H2a The relationship between class imbalance ratio and defective class recall shows a negative correlation which exists across all tested datasets and all types of classifiers. H2b The F1-scores of models which trained on datasets with lower feature noise exceed those of models which trained on datasets with higher noise under the same experimental conditions. H2c The SHAP-based interpretability analysis discovered that cyclomatic complexity and coupling metrics function as the primary defect predictors for most of the tested datasets.

## **6.3 Research Question 3 (RQ3)**

What gaps in current AI-based defect prediction research must be addressed to support broader industrial adoption?

### **6.3.1 Constructs and Operationalization**

Adoption gaps are operationalized through three dimensions which Section 5 describes. The first dimension assesses dataset representativeness through a comparison between benchmark dataset characteristics and documented industrial project profiles. The second dimension evaluates class imbalance remediation through the existence of a standardized protocol which handles skewed class distributions. The third dimension assesses model interpretability through the existence of feature-level explanations which practitioners can access during quality assurance processes. The synthesis evaluates each gap through existing evidence from reviewed literature and experimental results which RQ1 and RQ2 produced.

### **6.3.2 Rationale from Prior Work**

The literature consistently identifies three structural barriers to industrial adoption. The first barrier exists because benchmark datasets restrict research results from being applied to current agile and DevOps environments (Wahono, 2015; Hosseini et al., 2017). The second barrier exists because practitioners need evidence-based results to apply in their defect prediction model deployment on imbalanced production data yet no standardized class imbalance remediation protocol exists (Ni et al., 2022; Rajbahadur et al., 2021).

Third, practitioners need to understand high-performing ensemble and deep learning models because those systems produce outputs that remain difficult to comprehend (Jiarpakdee et al., 2022; Dam et al., 2019). The systematic review together with experimental results defines research gaps which require particular research work to complete each gap.

### **6.3.3 Testable Expectations (Hypotheses)**

H3a: No single class imbalance remediation technique produces consistently superior defective-class recall across all five benchmark datasets, confirming the absence of a generalizable standard.

H3b: SHAP-based explanations identify a common set of high-importance features across datasets, providing an empirical foundation for interpretability standards in defect prediction practice. The performance gap between benchmark evaluation results and cross-project evaluation results shows statistically significant difference which proves that industrial adoption becomes restricted through benchmark dependency.

## **7 Research Methodology**

### **7.1 Research Process**

The research study uses mixed-methods research design which combines a systematic literature review following PRISMA guidelines with a controlled evaluation of quantitative data and a qualitative analysis of barriers to practitioner adoption. The research process follows a structured workflow which maintains rigorous standards while providing transparent methods for testing scientific results at each research stage. The workflow operates through ten stages which proceed in order from problem identification to literature review through gap definition and research question development, followed by methodology development and data collection, data analysis, findings interpretation, conclusion development, and final reporting. Each stage informs the next — findings from the literature review shape the research questions, and experimental outcomes drive the interpretive analysis. The complete workflow is represented as a flowchart which appears in Figure 2.



Figure 2. Research Process Flowchart showing the sequential stages of the investigation.

## 7.2 Implementation Plan and Timeline (Weeks 1–15)

The study requires a 15-week implementation plan which starts in August 2026 and ends in November 2026. The activities of the project are divided into multiple phases which are scheduled according to the Gantt chart in Figure 3.

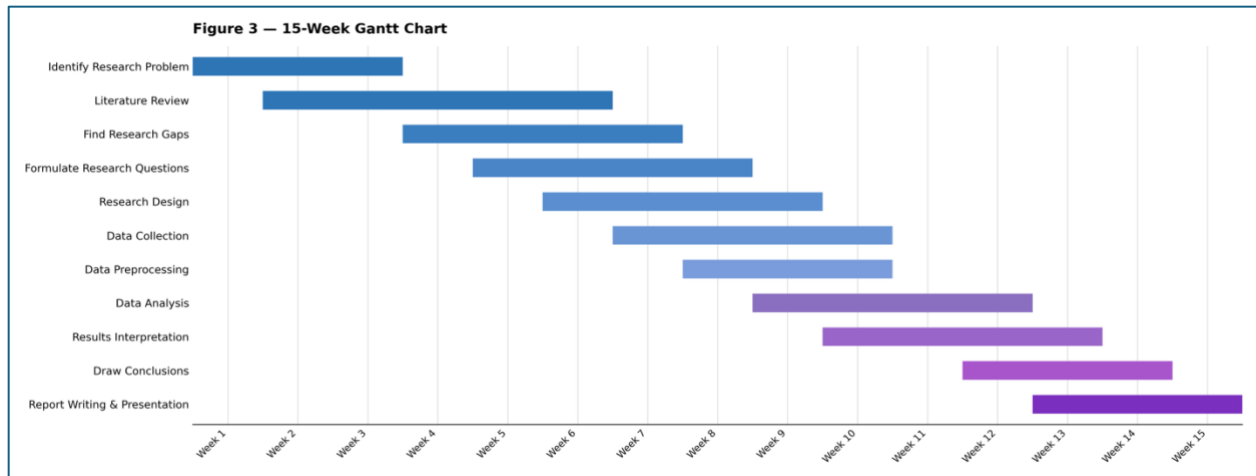


Figure 3. 15-Week Gantt Chart showing the phased research implementation timeline.

The first three weeks will begin with topic scoping and rapid literature scanning through selected databases. The team will establish the PRISMA protocol which will lead to conducting screening activities with the main article. The research team will conduct research gap analysis and develop research questions through their work in Week 6. The research team will complete their Research Questions milestone at the end of Week 6. The research team will establish the experimental environment between Weeks 7 and 9 which includes selecting datasets and building preprocessing pipelines and setting up Python environments. The research team will execute experiments between Weeks 10 and 12 which include model training and conducting stratified cross-validation and performance evaluation and SHAP analysis for RQ1 and RQ2. The research team will spend Weeks 13 and 14 on data analysis activities which will help them interpret research findings before they begin writing their proposal. The team will use Week 15 to complete their Research Proposal submission and prepare their oral presentation.

### 7.3 Research Design

The research study uses controlled comparative design to assess AI-based defect prediction models which are tested against a static analysis baseline on five standardized benchmark datasets. The entire research process applies the same preprocessing methods because this practice needs to maintain research validity and facilitate direct model evaluation. Researchers apply stratified ten-fold cross-validation to all models to create accurate performance assessments which reflect actual model capabilities.

The first research question gets answered through the multi-dataset framework whereas the second research question gets tested by manipulating class imbalance levels and feature noise conditions. The research tests each model configuration using all available datasets while reporting aggregated metrics with standard deviation values which show distribution across different testing conditions.

#### **7.4 Sampling and Data Selection**

The research used five datasets which were obtained from NASA MDP collection. The selection process required datasets to include standard Halstead and McCabe metrics together with documented binary defect labels and at least 200 instances. The five selected datasets enable researchers to conduct cross-dataset comparison because they contain different project sizes and defect densities and application domains.

#### **7.5 Experimental Procedure**

The following procedure is applied to every dataset-classifier combination. The preprocessing process includes Z-score normalization of feature values and column mean imputation for missing values and application of SMOTE oversampling to solve class imbalance issues. The classifiers Random Forest SVM Naive Bayes k-NN Gradient Boosting and LSTM are trained through stratified ten-fold cross-validation. Performance evaluation uses precision recall F1-score and AUC-ROC, which researchers calculate for all folds and present with standard deviation measurements. SHAP values were calculated for the best-performing model on each dataset to determine which five features had the highest predictive power and how they affected model predictions.

#### **7.6 Evaluation Metrics**

The main evaluation metrics include precision which measures the actual defects among flagged modules and recall which assesses the system's ability to detect actual defective modules and F1-score which calculates the precise and recall values through their harmonic mean and AUC-ROC which tests the system's ability to discriminate between different operational settings. The system tracks two secondary metrics of model training time and memory usage to evaluate its readiness for deployment.

## 7.7 Tools and Technologies

The researchers conduct all experiments using Python version 3.11. The study uses scikit-learn to implement classical ML classifiers, while LSTM models run on TensorFlow 2.x and SHAP analysis uses the shap library. The study uses SciPy and pandas for statistical analysis and Matplotlib and Seaborn for data visualization. The research will release all version-controlled code and preprocessing pipelines to the public after the study ends.

## 7.8 Research Validity

The research maintains internal validity through its use of ten-fold cross-validation with stratified sampling and its application of standardized data processing methods across all datasets and its use of pre-registered research hypotheses and its presentation of performance outcomes through standard deviation reporting. The external validity of the study exists within the boundaries of five NASA MDP benchmark datasets which the research uses as its data source while the researchers present their acknowledgment of the datasets' legacy status in Section 5.1. The research achieves reproducibility because all version-controlled code and preprocessing pipelines will be made available to the public according to the planned public release schedule.

## 8 Data Availability

All datasets used in this study are publicly accessible through established academic repositories without special permissions or institutional agreements. The NASA Metrics Data Program datasets which include CM1 and KC1 and KC3 and MC2 and PC1 are distributed through the PROMISE repository which resides at the West Virginia University Software Engineering Research Laboratory for non-commercial research purposes. The PROMISE repository additionally provides a range of open-source defect datasets which include Java and C and C++ projects that display binary defect indicators which were developed from version control histories and issue tracking systems.

The study will share its preprocessing code and experimental scripts and trained model configurations through a version-controlled repository which will become public after the final report submission according to open science standards. This procedure enables others to replicate the research work while they check the research results through independent assessment.

The legacy software project datasets which we used for our study do not accurately represent present-day agile and DevOps environments. The primary research gap which this limitation identifies appears in Section 5.1. Future research should prioritize access to industrial datasets through institutional partnerships. The analysis of all publicly available datasets raises no ethical concerns relating to personal data or privacy or informed consent.

### 8.1 Dataset Contents and Access

The five datasets — CM1, KC1, KC3, MC2, and PC1 — are drawn from the NASA Metrics Data Program collection, accessible through the PROMISE repository at West Virginia University. Each dataset includes two standard sets of software metrics: Halstead metrics and McCabe complexity metrics. Binary defect labels were derived from post-release defect tracking records. All five datasets are freely available for non-commercial academic research without institutional licensing requirements.

### 8.2 Example Data Record

Table 3 shows a sample record from the CM1 dataset which demonstrates the data structure used in this research. The table displays data for each software module through its row structure while showing Halstead and McCabe complexity metrics together with the defect label which uses a binary system.

Table 3. Example Data Record from the CM1 Dataset (NASA Metrics Data Program)

| Field     | Example Value  | Description                               |
|-----------|----------------|-------------------------------------------|
| Module ID | CM1_module_042 | Unique identifier for the software module |
| loc       | 384            | Lines of code (McCabe metric)             |
| v(g)      | 12             | Cyclomatic complexity                     |

|                  |     |                                               |
|------------------|-----|-----------------------------------------------|
| ev(g)            | 7   | Essential complexity                          |
| iv(G)            | 9   | Design complexity                             |
| lOCode           | 348 | Lines of executable code                      |
| lOComment        | 28  | Lines of comments                             |
| lOBlank          | 8   | Blank lines                                   |
| lOCodeAndComment | 4   | Lines containing both code and comment        |
| uniq_Op          | 21  | Unique operators (Halstead metric)            |
| defects          | 1   | Binary defect label: 1 = defective, 0 = clean |

Table 3 shows that every record contains complete static code metrics together with a binary defect identification system. The SMOTE-based remediation strategy described in Section 7.5 exists because of class imbalance which affects these datasets since defective modules make up about 10 percent of CM1.

### 8.3 Why These Datasets Were Selected

The five NASA MDP datasets were selected for four reasons. First, the established benchmark corpus includes multiple comparison points which permit researchers to assess their results against existing literature. The project types cover spacecraft instrumentation (CM1) storage management systems (KC1 and KC3) satellite software (MC2) and flight control software (PC1) which create different testing environments that follow the same assessment process. The datasets provide Halstead and McCabe metrics which create a unified measurement system for researchers to compare results across different datasets. The study follows open science principles because all research materials are available to the public without any access restrictions. The recognized limitation that these datasets originate from legacy software projects is explicitly acknowledged as Research Gap 1 in Section 5.1 and will be addressed during interpretation of findings.



## 9 Contribution of the Study

The five NASA MDP datasets were selected for four reasons. First, the established benchmark corpus includes multiple comparison points which permit researchers to assess their results against existing literature. The project types cover spacecraft instrumentation (CM1) storage management systems (KC1 and KC3) satellite software (MC2) and flight control software (PC1) which create different testing environments that follow the same assessment process. The datasets provide Halstead and McCabe metrics which create a unified measurement system for researchers to compare results across different datasets. The study follows open science principles because all research materials are available to the public without any access restrictions. The recognized limitation that these datasets originate from legacy software projects is explicitly acknowledged as Research Gap 1 in Section 5.1 and will be addressed during interpretation of findings.

The PRISMA-based systematic literature review creates a systematic and transparent overview of existing knowledge about AI software defect prediction through its analysis of fifteen studied articles which have undergone peer review. The study demonstrates its ability to be replicated through selection criteria which reveal to researchers which components of the study they will include and which they will exclude (Page et al., 2021). The results show which techniques succeed while they also explain the reasons behind the failure of particular methods during particular situations. The research formulation of three research questions which include testable hypotheses creates a research framework that other researchers can use in their studies of different industrial environments and cross-project conditions.

The research outputs are intended to provide software quality assurance practitioners with evidence-based guidance on selecting AI tools, applying imbalance remediation strategies, and integrating AI defect prediction results reliably into code review workflows. The interpretability analysis addresses the established trust gap which exists between technical model performance metrics and user confidence by showing how SHAP-based explanations enable professionals to use AI-based recommendations in their work (Jiarpakdee et al., 2022; Dam et al., 2019). The researchers plan to submit their findings to a Q1 or Q2 software engineering journal which includes Empirical Software Engineering and the Journal of Systems and Software.

## **10 Ethical Considerations**

### **10.1 Data Licensing and Intellectual Property**

All datasets are obtained from publicly available academic repositories under open-access licensing for non-commercial research use. The NASA MDP and PROMISE datasets are available to anyone without requiring them to sign individual licensing contracts. The study materials contain no proprietary software code or commercially sensitive information or personally identifiable information. The research team gives appropriate credit to all datasets and software libraries according to the specific license requirements which govern their use. The research project will distribute its experimental code as open-source software after the study ends because this method promotes scientific transparency and reproducible research results and it ensures responsible research practices.

### **10.2 Responsible Use of AI Models**

The AI models in this research study serve academic research purposes only within a controlled testing environment. The AI models in this study require experimental testing because they cannot be used in actual software engineering systems and their predictions do not impact either software products or end users or organizational decisions. The research employs SHAP-based interpretability analysis because it enables model results to be understood through clear model outputs which protect users from making choices based on unknown results that lack expert evaluation (Dam et al., 2019). The training data contains potential biases, which the proposal recognizes and discusses because of class imbalance and benchmark datasets that represent historical data and software domains with a restricted user base. The study does not need institutional ethical review board approval because it does not involve human participants or gather behavioral data or process personally identifiable information. The research will present all findings, including negative results and observed limitations, according to principles that define research integrity

.

## 11 Conclusion

The AI models in this research study serve academic research purposes only within a controlled testing environment. The AI models in this study require experimental testing because they cannot be used in actual software engineering systems and their predictions do not impact either software products or end users or organizational decisions. The research employs SHAP-based interpretability analysis because it enables model results to be understood through clear model outputs which protect users from making choices based on unknown results that lack expert evaluation (Dam et al., 2019). The training data contains potential biases, which the proposal recognizes and discusses because of class imbalance and benchmark datasets that represent historical data and software domains with a restricted user base. The study does not need institutional ethical review board approval because it does not involve human participants or gather behavioral data or process personally identifiable information. The research will present all findings, including negative results and observed limitations, according to principles that define research integrity.

The research plan consists of four components which include the 15-week implementation plan and the controlled experimental design and the SHAP-based interpretability framework and the ethical governance structure. The expected outputs include a cross-system performance evaluation, a synthesized literature review, and an interpretability analysis designed to inform practitioner decision-making.

The research establishes evidence that supports real-world software engineering deployment of AI-based quality assurance systems by demonstrating both the reliable performance of AI tools and the trust-building effect of interpretability.

## References

- Albattah, W., & Alzahrani, M. (2024). Software defect prediction based on machine learning and deep learning techniques: An empirical approach. *Software Engineering Research*, 5(4), 86–102. <https://doi.org/10.3390/ser5040086>
- Dam, H. K., Tran, T., Pham, T. T. M., Ng, S. W., Grundy, J., & Ghose, A. (2019). Explainable software analytics. In *Proceedings of the 41st International Conference on Software Engineering: New Ideas and Emerging Results* (pp. 53–56). IEEE. <https://doi.org/10.1109/ICSE-NIER.2019.00022>
- Guo, Y., Zhao, Y., & Wang, Q. (2023). An empirical study of deep learning models for software defect prediction. *Empirical Software Engineering*, 28(3), 64. <https://doi.org/10.1007/s10664-022-10263-w>
- Hall, T., Beecham, S., Bowes, D., Gray, D., & Counsell, S. (2012). A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering*, 38(6), 1276–1304. <https://doi.org/10.1109/TSE.2011.103>
- Hosseini, S., Turhan, B., & Gunarathna, D. (2017). A systematic literature review and meta-analysis on cross project defect prediction. *IEEE Transactions on Software Engineering*, 45(2), 111–145. <https://doi.org/10.1109/TSE.2017.2770189>
- Jiarpakdee, J., Tantithamthavorn, C., & Grundy, J. (2022). Automated feature selection for explaining and predicting software defect prediction models. *Journal of Systems and Software*, 184, 111170. <https://doi.org/10.1016/j.jss.2021.111170>
- Li, Z., Jing, X. Y., & Zhu, X. (2018). Progress on approaches to software defect prediction. *IET Software*, 12(3), 161–175. <https://doi.org/10.1049/iet-sen.2017.0148>
- Malhotra, R. (2015). A systematic review of machine learning techniques for software fault prediction. *Applied Soft Computing*, 27, 504–518. <https://doi.org/10.1016/j.asoc.2014.11.023>
- Ni, A., Li, M., & Peng, X. (2022). Revisiting the impact of class rebalancing techniques on defect prediction. In *Proceedings of the 30th ACM Joint European Software Engineering*

- Conference and Symposium on the Foundations of Software Engineering (pp. 370–382). ACM. <https://doi.org/10.1145/3540250.3549145>
- Ni, C., Wang, S., & Lyu, M. R. (2023). Just-in-time defect prediction on JavaScript projects: A replication study. *Information and Software Technology*, 159, 107191. <https://doi.org/10.1016/j.infsof.2023.107191>
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., & Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372, n71. <https://doi.org/10.1136/bmj.n71>
- Pornprasit, C., & Tantithamthavorn, C. (2023). Jitline: A simpler, better, faster method for just-in-time defect prediction. *IEEE Transactions on Software Engineering*, 49(1), 98–116. <https://doi.org/10.1109/TSE.2021.3073176>
- Rajbahadur, G. K., Wang, S., Kamei, Y., & Hassan, A. E. (2021). The impact of using regression models to build defect classifiers. *Empirical Software Engineering*, 26(4), 76. <https://doi.org/10.1007/s10664-021-09972-0>
- Wahono, R. S. (2015). A systematic literature review of software defect prediction. *Journal of Software Engineering*, 1(1), 1–16.
- Zeng, Z., Zhang, Y., Zhang, H., & Zhang, W. (2021). Deep just-in-time defect prediction on JavaScript projects. In *Proceedings of the 43rd International Conference on Software Engineering* (pp. 1759–1770). IEEE. <https://doi.org/10.1109/ICSE43902.2021.00160>